Adunarea/scăderea

- sintaxa

`add` *op1*`,`*op2* // op1 += op2

`sub` *op1*`,`*op2* // op1 -= op2

- sunt posibile aceleași combinații de operanzi ca în cazul atribuirii
- indicatorii de condiții primesc valori conform rezultatului operației

Exemple

```
add eax,ebx
add dl,3
add si,[ecx]
add [eax+edi],ebp
add byte ptr [esi+10],14
add word ptr [esi+10],14
add dword ptr [esi+10],14
```

Operații cu și fără semn (1)

```
unsigned int a;
int b;
_asm add a,20;
_asm add b,20;
```

- rezultatul poate fi incorect (depășire)
- trebuie testat indicatorul Carry (pentru `a`), respectiv Overflow (pentru `b`)

Operații cu și fără semn (2)

- problemă
  - operanzii `a` și `b` sunt adrese de memorie
  - procesorul nu știe care este număr cu semn și care fără semn
  - deci care indicator trebuie setat: C sau O?
- răspuns
  - procesorul setează toți indicatorii
  - programatorul trebuie să-l testeze pe cel potrivit

Incrementare/decrementare

- sintaxa

`inc` *op* // op++

`dec` *op* // op--

- operandul poate fi
  - registru (orice dimensiune)
  - locație de memorie - dimensiunea trebuie precizată explicit

Adunare/scădere cu transport

- sintaxa

`adc` *op1*`,`*op2* // op1=op1+op2+Carry

`sbb` *op1*`,`*op2* // op1=op1-op2-Carry

- la adunare/scădere participă și valoarea anterioară a indicatorului Carry
- rol - operații cu numere care depășesc dimensiunea unei variabile

Înmulţirea

- sintaxa

`mul` *op* // numere fără semn

`imul` *op* // numere cu semn

- este indicat explicit un singur operand
  - poate fi registru sau locație de memorie
- al doilea este implicit și depinde de dimensiunea primului

Operanzi (1)

- operandul explicit nu poate fi o constantă
- dacă operandul explicit este o locație de memorie, trebuie precizată și dimensiunea sa
- destinația rezultatului este tot implicită și necesită o dimensiune dublă față de operanzi

Operanzi (2)

dimensiune operand explicit	operand implicit	destinaţie rezultat
1 octet	`al`	`ax`
2 octeți	`ax`	`(dx,ax)`
4 octeți	`eax`	`(edx,eax)`

Exemple

```
mul ebx  // eax·ebx→(edx,eax)
mul cx  // ax·cx→(dx,ax)
mul al  // se ridică al la pătrat
mul dword ptr [esi]
  // eax·[esi]→(edx,eax)
  // operanzi pe 4 octeți
imul cx  // operanzi cu semn
```

Împărţirea

- sintaxa

`div` *op* // numere fără semn

`idiv` *op* // numere cu semn

- este indicat explicit doar împărțitorul
  - poate fi registru sau locație de memorie
- deîmpărțitul este implicit și depinde de dimensiunea împărțitorului

Operanzi (1)

- împărțitorul nu poate fi o constantă
- dacă împărțitorul este o locație de memorie, trebuie precizată și dimensiunea sa
- două rezultate: câtul și restul
- destinațiile acestora sunt tot implicite

Operanzi (2)

dimensiune împărţitor	deîmpărţit	cât	rest
1	`ax`	`al`	`ah`
2	`(dx,ax)`	`ax`	`dx`
4	`(edx,eax)`	`eax`	`edx`

Împărțire la 0

- dacă împărțitorul are valoarea 0, se produce o eroare la execuție
  - la fel ca în orice limbaj
- problema poate apărea și în alte situații
  - obținem același efect, deși împărțitorul este nenul

Exemplu

```
_asm {
  mov eax,1
  mov edx,1
  mov ebx,1
  div ebx
  }
```

- secvența de mai sus produce eroare

De ce?

- împărțitorul este `ebx` - 4 octeți
- deci deîmpărțitul este (`edx`,`eax`)
- iar câtul trebuie depus în `eax`
- dar valoarea câtului este $2^{32}+1$ - prea mare pentru a încăpea în `eax`
  - similar cu împărțirea la 0
  - produce aceeași eroare

Când apare eroarea?

- dacă și numai dacă jumătatea mai semnificativă a deîmpărţitului este mai mare sau egală cu împărţitorul
- exemplul anterior

```
div ebx
```

  – jumătatea mai semnificativă a deîmpărţitului: `edx` (= 1 ≥ `ebx`)

Apare problema în limbajul C?

- nu
- se împart întotdeauna operanzi de acelaşi tip
- împărţitorul se alege de dimensiune egală cu a tipului de date folosit
- deîmpărţitul are implicit dimensiune dublă
  - dar jumătatea sa mai semnificativă va primi valoarea 0

Instrucţiuni pe biți

Instrucţiuni booleene

- implementează funcțiile booleene elementare
  - unară: NOT
  - binare: AND, OR, XOR
- instrucțiunile au aceleași nume ca și funcțiile pe care le implementează
- execuție: se aplică funcția, în paralel, pe toți biții operanzilor

Exemple

```
not eax
and bx,16
or byte ptr [edx],100
xor [esi],ecx
test al,ah // va fi discutată ulterior
```

- și aici, primul/singurul operand este și destinația rezultatului

Utilitate

- există operatori similari și în limbajul C
- rar utilizați
- mai ales acolo unde este nevoie de tratarea individuală a unor biți
  - lucru cu măști
- în practică, doar funcția AND se pretează la așa ceva

Lucrul cu măşti (1)

- problemă: trebuie să testăm bitul de pe poziţia 3 din registrul `ax`
  - pentru testare putem folosi indicatorul Zero al procesorului
  - dar acesta ne poate spune doar dacă întregul operand (`ax`) are valoarea 0 sau nu
- soluție: toți ceilalți biți din `ax` (cu excepția poziției 3) primesc valoarea 0

Lucrul cu măști (2)

- astfel, registrul `ax` are valoarea 0 dacă și numai dacă bitul de pe poziția 3 este 0
- deci indicatorul Zero ne va furniza informația corectă

`and ax,8` // deoarece $8 = 2^3$

- al doilea operand este o constantă - *mască*
  - bitul 1 doar pe poziția 3 - rămâne nemodificat

Lucrul cu măşti (3)

- ceea ce ne doream era o testare a valorii unui bit
  - nedistructiv
- în schimb, toţi ceilalţi biţi din `ax` se pierd
- ar trebui ca mai întâi să salvăm valoarea din `ax` în alt registru
  - ineficient

Lucrul cu măşti (4)

- instrucțiunea `test` calculează tot funcția booleană AND
  - dar nu scrie rezultatul nicăieri - nedistructiv
  - doar setează indicatorii de condiții în același mod

`test ax,8` // `ax` nu este modificat

// acum putem testa indicatorul Zero

Instrucţiuni de deplasare

- sintaxă

`shl` *op1*`,`*op2* // deplasare spre stânga

`shr` *op1*`,`*op2* // deplasare spre dreapta

- *op1* - operandul asupra căruia se aplică deplasarea
- *op2* - indică numărul de poziții pe care se face deplasarea

Operanzi

- primul operand poate fi
  - registru
  - locație de memorie (trebuie indicată explicit dimensiunea sa)
- al doilea operand poate fi
  - constantă (1 sau mai mult)
  - registrul `cl` - singurul permis

Implementare

- fiecare bit din *op1* este mutat spre stânga/dreapta cu *op2* poziții
- biții care "ies" sunt memorați, pe rând, în indicatorul Carry
  - evident, numai ultimul este păstrat
- biții care "intră" au toți valoarea 0

Utilitate practică (1)

- pentru numere fără semn

✓ deplasare spre stânga cu *n* poziții - echivalent cu înmulțirea cu 2^n

✓ deplasare spre dreapta cu *n* poziții - echivalent cu împărțirea la 2^n

Utilitate practică (2)

- dar pentru numere cu semn?

✓ deplasare spre stânga - nu se poate

✓ deplasare spre dreapta

  – pentru a realiza o împărțire corectă la 2^n, bitul de semn trebuie să rămână nemodificat

  – și în același timp să se deplaseze și spre dreapta

  – deci biții care "intră" nu mai au valoarea 0, ci sunt egali cu bitul de semn

Deplasare aritmetică

- sintaxă

`sar op1,op2`

- operanzii au aceleași roluri ca anterior
- dar instrucțiunea realizează de fapt împărțire la 2^n pentru numere cu semn

Exemple

```
shl eax,1
shl byte ptr [ebx+ecx],3
shr dx,cl
sar eax,2
```